

2D Geometry Editor 代码说明

一 . main.py 代码说明

1. 文件作用

main.py 是程序入口文件，用于初始化 Qt 应用并启动主窗口，是整个系统运行的起点。

2. 核心流程

```
app = QApplication(sys.argv)
window = MainWindow()
window.show()
sys.exit(app.exec_())
```

3. 关键说明

(1) 创建应用对象

QApplication 用于管理整个 GUI 程序的事件循环，是 PyQt 程序的基础。

(2) 创建主窗口

MainWindow 负责构建界面，包括按钮和绘图画布。

(3) 显示窗口

`show()` 用于将窗口显示在屏幕上。

(4) 进入事件循环

`app.exec_()` 使程序进入监听状态，响应用户的鼠标和键盘操作。

4. 执行流程总结

`main.py` 启动

- 创建应用
- 创建窗口
- 显示界面
- 进入事件循环

二 . `main_window.py` 代码解释

1. 文件作用

`main_window.py` 负责构建程序的主界面，包括窗口布局、按钮区域以及绘图画布（Canvas）的加载，并将用户操作与功能逻辑连接起来。

2. 界面结构组成

该界面主要由两部分构成：

- 顶部：功能按钮区（Clear / Save）
 - 底部：绘图画布（Canvas）
-

3. 核心代码结构

```
self.canvas = Canvas()
```

```
central_widget = QWidget()
```

```
self.setCentralWidget(central_widget)
```

```
main_layout = QVBoxLayout(central_widget)
```

说明：

- Canvas(): 创建绘图区域（核心绘图逻辑在此）
 - central_widget: 主窗口容器
 - QVBoxLayout: 垂直布局，使按钮在上、画布在下
-

4. 按钮功能绑定

```
clear_button = QPushButton("Clear")
```

```
clear_button.clicked.connect(self.clear_canvas)
```

```
save_button = QPushButton("Save")
save_button.clicked.connect(self.canvas.save_to_json)
```

说明：

- Clear 按钮：清空当前画布内容
 - Save 按钮：调用 Canvas 中的 JSON 保存功能
 - clicked.connect(): 将按钮与函数绑定，实现交互
-

5. 布局组织方式

```
button_widget = QWidget()
button_widget.setLayout(button_layout)
```

```
main_layout.addWidget(button_widget)
main_layout.addWidget(self.canvas)
```

说明：

- 使用 QWidget 包装按钮区域，避免布局被压缩
 - 按钮区域 + 画布区域依次加入主布局
-

6. 清空功能逻辑

```
def clear_canvas(self):
```

```
self.points.clear()
self.lines.clear()
self.curves.clear()
self.temp_points = []
self.update()
```

说明：

该函数用于重置画布状态：

- 清空点、线、曲线数据
 - 清空临时绘制点
 - update()：刷新界面
-

7. 总体作用总结

该文件主要完成三件事：

- 构建主窗口界面结构
- 绑定按钮与功能逻辑
- 连接 UI 与绘图核心（Canvas）

三 . canvas.py 代码解释

1. 文件作用

Canvas 是整个项目的核心绘图与交互模块，负责处理用户的鼠标操作、几何数据管理以及界面绘制，包括点、线、曲线的生成与显示。

2. 数据结构

```
self.points = []
```

```
self.lines = []
```

```
self.curves = []
```

```
self.temp_points = []
```

说明：

- points：存储所有点
 - lines：存储直线
 - curves：存储贝塞尔曲线
 - temp_points：用于临时记录生成曲线的 3 个点
-

3. 鼠标点击事件（核心交互）

功能：创建点 / 连接线 / 生成曲线 / 拖拽

```
def mousePressEvent(self, event):
```

(1) 右键删除线

通过计算“点到线的距离”实现删除功能：

- 如果点击位置靠近线段 (<8 像素)
 - 删除该线段
-

(2) 左键逻辑 (核心)

✓ 点击已有点

- 将点加入 temp_points
- 每 3 个点自动生成一条二次贝塞尔曲线
- 支持：
 - 拖拽点
 - 连线功能 (两点生成线)

✓ 点击空白区域

- 创建新的 Point 对象并加入 points 列表
-

4. 点到线距离计算

```
def point_to_line_distance(self, px, py, x1, y1, x2, y2):
```

作用：

判断鼠标是否接近线段，用于删除操作。

原理：

- 计算点到线段的垂直距离
 - 若在线段范围内，则返回最近距离
-

5. 查找点函数

```
def find_point(self, x, y):
```

功能：

判断用户点击是否命中已有点或曲线控制点。

逻辑：

- 优先检测普通点
 - 再检测曲线控制点
 - 距离小于 8 像素则视为选中
-

6. 绘图函数（核心渲染）

```
def paintEvent(self, event):
```

功能：

负责所有图形的绘制，包括：

- 网格背景

- 线段
 - 曲线
 - 点
-

(1) 网格绘制

使用等间距网格增强绘图参考性：

- 横线 + 竖线
 - 间隔 25 像素
-

(2) 线段绘制

遍历 lines，使用 drawLine 进行绘制。

(3) 曲线绘制

使用 QPainterPath 实现二次贝塞尔曲线：

- start → 起点
 - control → 控制点
 - end → 终点
-

(4) 点绘制

- 普通点： 黑色
 - 选中点： 红色
 - 使用 `drawEllipse` 绘制圆点
-

7. 鼠标拖拽逻辑

```
def mouseMoveEvent(self, event):
```

功能：

- 当拖动点时更新其坐标
 - 实现实时拖拽效果
 - 调用 `update()` 刷新画面
-

8. 鼠标释放

```
def mouseReleaseEvent(self, event):
```

功能：

- 结束拖拽状态
 - 清空 `dragging_point`
-

9. JSON 保存功能

```
def save_to_json(self):
```

功能：

将当前场景保存为 JSON 文件。

保存内容：

- points（点坐标）
 - lines（通过索引保存连接关系）
-

10. 总体总结

Canvas 是整个系统的核心模块，负责：

- 几何数据管理（点、线、曲线）
- 用户交互（点击、拖拽、删除）
- 图形渲染（网格 + 图形绘制）
- 数据持久化（JSON 保存）

四 . Point 类代码解释

1. 文件作用

Point 类用于表示二维空间中的一个**点对象**，是整个几何编辑器中最基础的数据结构，用于存储点的坐标信息。

2. 代码结构

```
class Point:

    def __init__(self, x, y):

        self.x = x

        self.y = y
```

3. 逐部分解释

(1) 类定义

```
class Point:
```

作用：

- 定义一个“点”的数据类型
 - 用于封装二维坐标 (x, y)
-

(2) 初始化函数

```
def __init__(self, x, y):
```

作用：

- 构造函数
- 创建 Point 对象时自动调用
- 接收两个参数：

- x: 横坐标
 - y: 纵坐标
-

(3) 属性赋值

`self.x = x`

`self.y = y`

作用：

- 将输入的坐标保存为对象属性
 - 使每个 Point 实例都拥有自己的位置数据
-

4. 数据结构作用

Point 在整个项目中的作用是：

- 表示图形中的顶点
 - 用于构建线 (Line)
 - 用于构建曲线 (Curve)
 - 支持拖拽移动
-

5. 总体总结

Point 类是整个几何系统的基础数据单元，用于表示二维空

间中的位置，是线段和曲线构造的基础组成部分。

五 . Line 类代码解释

1. 文件作用

Line 类用于表示二维空间中的**线段对象**，由两个点组成，是构成图形连接关系的重要数据结构。

2. 代码结构

```
class Line:
    def __init__(self, start_point, end_point):
        self.start = start_point
        self.end = end_point
```

3. 逐部分解释

(1) 类定义

```
class Line:
```

作用：

- 定义“线段”这一几何对象
- 用于连接两个 Point 点

(2) 初始化函数

```
def __init__(self, start_point, end_point):
```

作用：

- 构造函数
- 创建 Line 对象时自动调用
- 接收两个参数：
 - start_point: 起点
 - end_point: 终点

(3) 属性赋值

```
self.start = start_point
```

```
self.end = end_point
```

作用：

- 保存线段的两个端点
- 用于后续绘制、删除、计算距离等操作

4. 在项目中的作用

Line 类主要用于：

- 连接两个点形成线段
 - 在画布中进行绘制
 - 支持删除操作（通过点到线距离判断）
 - 构建几何结构关系
-

5. 总体总结

Line 类是二维几何系统中的基础连接结构，用于表示两个点之间的直线关系，是构建复杂图形的基本单元。

六 . Curve 类代码解释

1. 文件作用

Curve 类用于表示二维空间中的二次贝塞尔曲线对象，由三个点构成，是本项目中实现曲线绘制功能的核心数据结构。

2. 代码结构

```
class Curve:
```

```
    def __init__(self, start, control, end):  
        self.start = start
```



```
self.control = control
```

```
self.end = end
```

3. 逐部分解释

(1) 类定义

```
class Curve:
```

作用：

- 定义曲线数据结构
 - 用于表示二次贝塞尔曲线
-

(2) 初始化函数

```
def __init__(self, start, control, end):
```

作用：

- 构造函数
- 创建 Curve 对象时自动调用
- 接收三个点：
 - start: 起点
 - control: 控制点（决定曲线弯曲方向）

- end: 终点
-

(3) 属性赋值

`self.start = start`

`self.control = control`

`self.end = end`

作用：

- 保存曲线的三个关键控制点
 - 用于后续绘制贝塞尔曲线（`QPainterPath`）
 - 控制曲线形状与弯曲程度
-

4. 在项目中的作用

`Curve` 类用于：

- 生成二次贝塞尔曲线
 - 通过三个点控制曲线形状
 - 在 `Canvas` 中进行绘制（`QPainterPath`）
 - 支持动态交互（拖动控制点改变曲线形状）
-

5. 总体总结

Curve 类是本项目中实现曲线功能的核心结构，通过“起点 + 控制点 + 终点”定义曲线形态，实现类似 CAD/绘图软件的曲线效果。